计算机系统原理

姓名：江李阳, 王健一, 邱泽辉

学号：202400460042, 202400460037,202400460046

专业：密码科学与技术

分工：1,1,1

日期：April 6, 2026

Contents

1 实验一 **2**
1.1 实验环境 2
1.2 实验目的 2
1.3 实验原理 2
1.4 实验设计 3
1.5 实验过程 3
1.6 实验结果 4
1.7 实验结果分析 6
1.8 实验结论 7

1 实验一

1.1 实验环境

- 硬件环境:
  - 处理器:13th Gen Intel(R) Core(TM) i5-13420H(12CPUs), 2.1GHz
  - 内存:16384MB RAM
- 软件环境:
  - 操作系统:Windows11
  - VMware Workstation Pro 17:Ubuntu 22.04 LTS 系统,7.8GB 内存,4 处理器,300GB 硬盘
  - Arch Linux x86_64(内核:Linux 6.6.87.2-microsoft-standard-WSL2)
    - L1d cache:288KiB(6 instances)
    - L1i cache:192 KiB(6 instances)
    - L2 cache:7.5 MiB(6 instances)
    - L3 cache:11MiB(1 instances)

1.2 实验目的

- 学习矩阵乘法程序优化: 在考虑数据大小, 线程数, 硬件配置, 程序算法等的前提下考虑矩阵乘法程序的优化.
- 理解 Cache 机制: 通过改变访存方式, 验证时间局部性与空间局部性对程序执行效率的影响. 进而最终理解 Cache 对程序执行效率的巨大影响.
- 探索并行计算机制: 利用多线程与指令级并行 (SIMD/循环展开), 分析硬件资源并行优化带来的加速比.
- 掌握跨平台工程能力: 学习交叉编译和 QEMU 模拟器的使用, 学习掌握 RISC-V 工具链的基本使用方法. 学习 RISC-V Linux 内核编译并通过 QEMU 运行.

1.3 实验原理

矩阵乘法 $C_{m\times n} = A_{m\times k} \times B_{k\times n}$, 要求第一个矩阵的列数等于第一个矩阵的行数, 一般的实现方法为 3 层循环实现, 最内层最内层循环分别遍历 A 的行和 B 的列, 其算法时间复杂度为 $O(N^3)$. 在现代计算机体系结构下, 制约其性能的核心因素通常不是计算速度, 而是存储层级:

- 空间局部性: 当访问矩阵 B 时将固定列而遍历行, 对于按行优先存储的编程语言, 这将跳跃访问内存地址, 空间局部性差.
- 当矩阵的规模较大而超过 L/L2Cache 的大小时, 内存 MISS 率将大大提高, 程序执行效率低下. 通过矩阵分块, 使分块后的矩阵能完整装入 L1/L2Cache, 将减少对主存的访问.
- 多核并行: 通过将维度任务划分, 平衡 CPU 的核心负载, 避免因负载不均导致的程序整体效率低.

1.4 实验设计

本实验一共考虑并设计了 10 个版本的矩阵乘法代码, 每个版本都在不同矩阵大小规模和线程数的条件下运行并测量结果, 其中实验设计如下:

Table 1: 矩阵乘法优化版本迭代说明

版本分类	核心技术	优化逻辑说明
基础版 [版本 0]	Naive	最原始的 i-j-k 嵌套, 访存步长较大,Cache Miss 极高
访存优化 [版本 1]	IKJ 重排	改变循环顺序, 使内层循环对 B 和 C 均为连续内存访问
访存优化 [版本 2]	矩阵转置	预先转置矩阵 B, 将纵向访问变为横向连续访问
访存优化 [版本 3]	分块 (Blocking)	利用分块技术强行提高时间局部性, 适配 Cache 容量
并行优化 [版本 4]	多线程 +IKJ	将矩阵行拆分给多个线程, 并行执行最高效的重排算法
并行优化 [版本 5]	多线程 + 分块	结合线程并行与 Cache 优化, 处理超大规模矩阵
并行优化 [版本 8]	全优化 (并行 + 转置)	融合转置带来的访存优势与多线程的计算优势
指令优化 [版本 6/9]	循环展开/向量化	减少循环跳转开销, 利用编译器自动向量化压榨流水线
指令优化 [版本 10]	手动 AVX2(仅 WSL)	手动调用 Intel 指令集, 单指令多数据并行处理

1.5 实验过程

编写对应各个版本矩阵乘代码, 考虑到模拟环境性能较差, 运行速度较慢, 所以首先在宿主机 WSL2(Arch Linux) 上运行测试 (考虑到不同版本的代码在不同平台测试结果相差较大, 但在仙女沟通平台应呈现出一致的趋势). 实验过程截图如图:

```
[root@LYJ code]# ./matmul_all
==================== Matrix Multiplication Benchmark ====================
CPU: 6 Cores | High Precision Timer (Nanosecond)
Sizes: 32/64/128/256/512/1024/2048
=========================================================================
Ver          Size    Th      Block       Time(s)      GFLOPS       Note
v0           32      1       -           0.000021     3.14         Baseline
v1           32      1       -           0.000009     7.11         IKJ
v2           32      1       -           0.000024     2.72         Transpose
v3           32      1       32          0.000009     6.93         Block
v3           32      1       64          0.000009     7.10         Block
v3           32      1       128         0.000009     7.17         Block
v3           32      1       256         0.000009     7.09         Block
v4           32      1       -           0.009365     0.01         MT-IKJ
v4           32      2       -           0.000284     0.23         MT-IKJ
v4           32      4       -           0.000395     0.17         MT-IKJ
v4           32      6       -           0.000510     0.13         MT-IKJ
v4           32      8       -           0.000439     0.15         MT-IKJ
v4           32      12      -           0.000668     0.10         MT-IKJ
v5           32      1       32          0.000118     0.56         MT-Block
v5           32      1       64          0.000118     0.55         MT-Block
v5           32      1       128         0.000119     0.55         MT-Block
v5           32      1       256         0.000150     0.44         MT-Block
v5           32      2       32          0.000144     0.45         MT-Block
v5           32      2       64          0.000120     0.54         MT-Block
v5           32      2       128         0.000128     0.51         MT-Block
```

Table 2: 实验一运行过程截图 (宿主机)

实验代码在宿主机上成功运行得到准确数据后, 迁移至 VMware 的 ubuntu 虚拟机上, 在 QEMU 环境上测试运行。由于软件模拟导致性能远小于在宿主机上运行, 因此适当减小了实验数据, 并删去了 X86 特有的 AVX2 指令集, 实验过程截图如图:

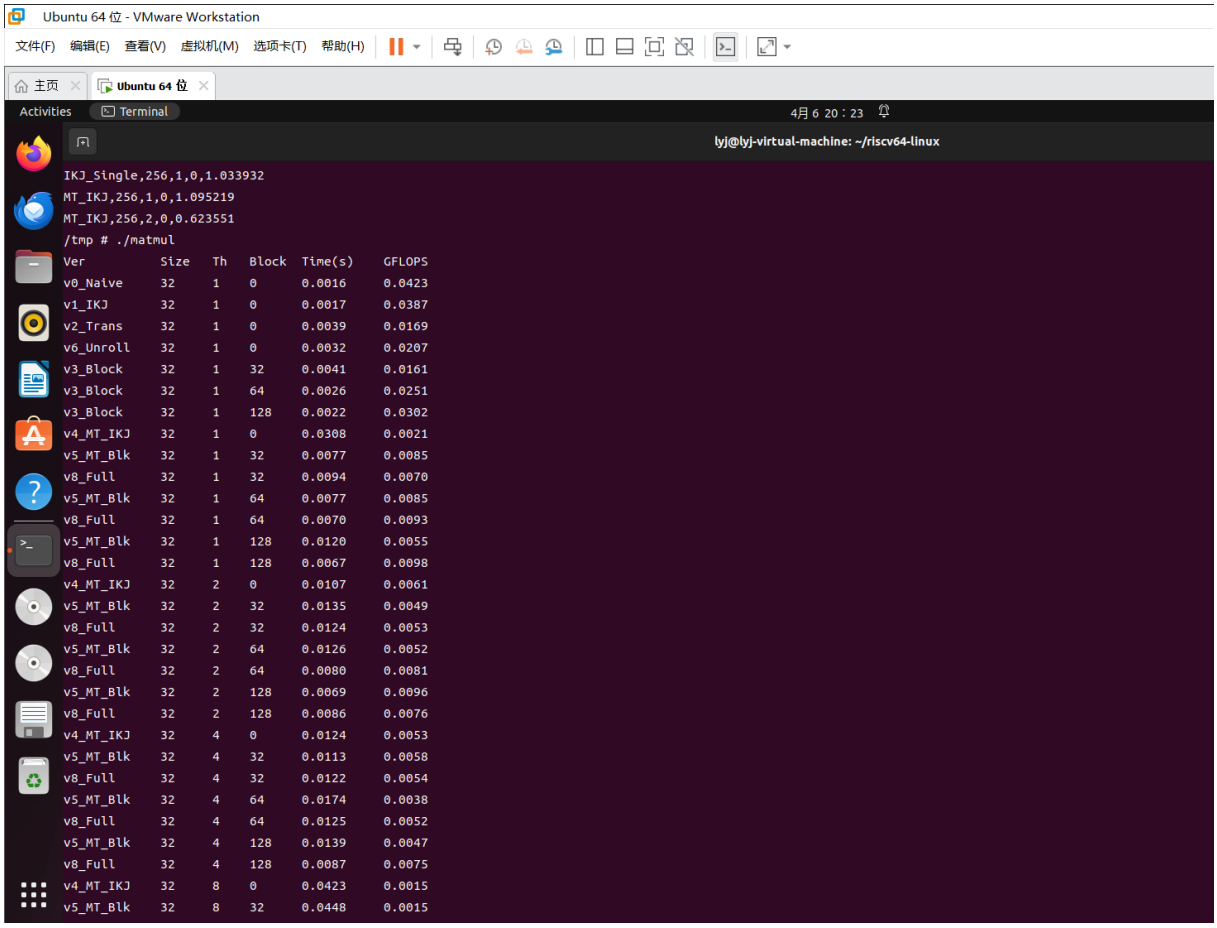

Table 3: 实验一运行过程截图 (VMware)

1.6 实验结果

不同优化版本的矩阵乘法代码在各种测试条件下 (输入规模大小、并行线程数、分块大小) 在 WSL2(Arch Linux) 上运行结果绘制成折线图如图:

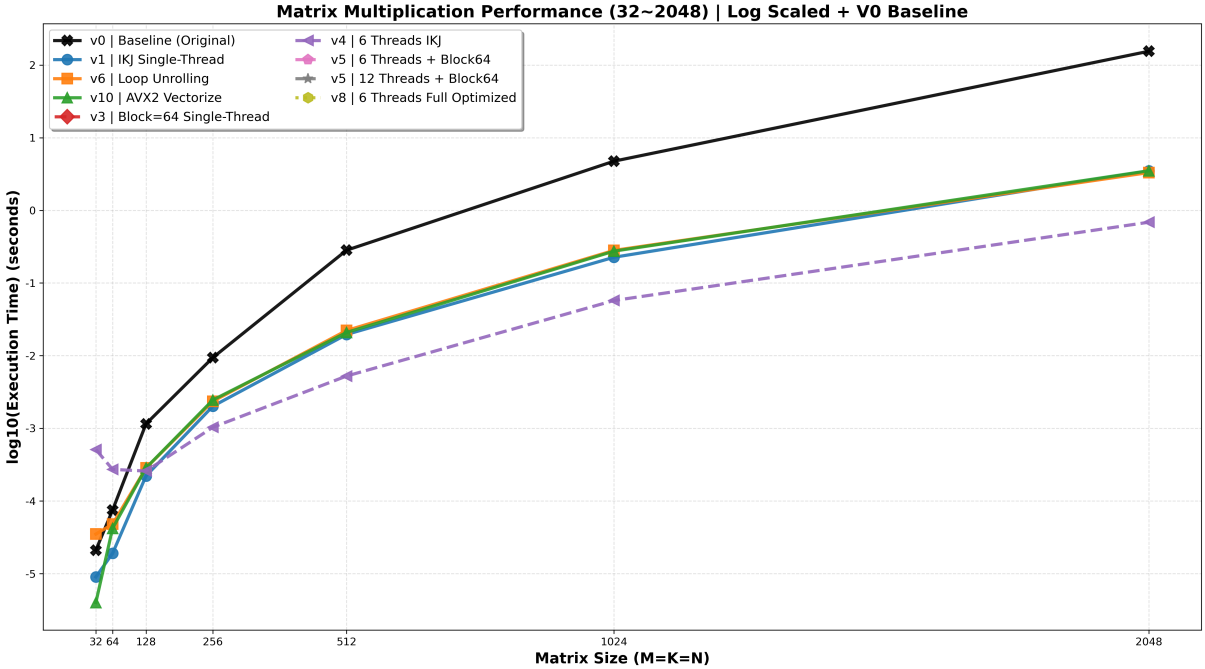

Figure 1: 实验一运行结果折线图 (宿主机)

其中纵坐标为对应运行时间取对数。相应的修改代码在 QEMU 模拟环境上运行结果绘制成折线图如图:

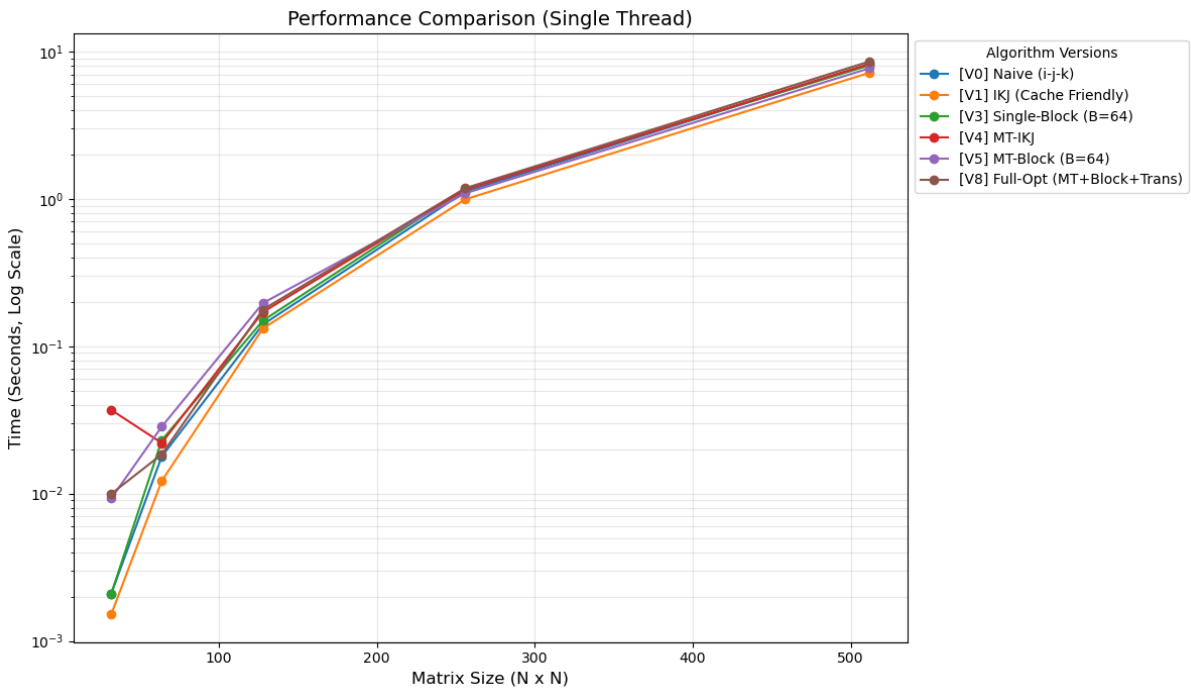

Figure 2: 实验一运行结果折线图 (VMware)

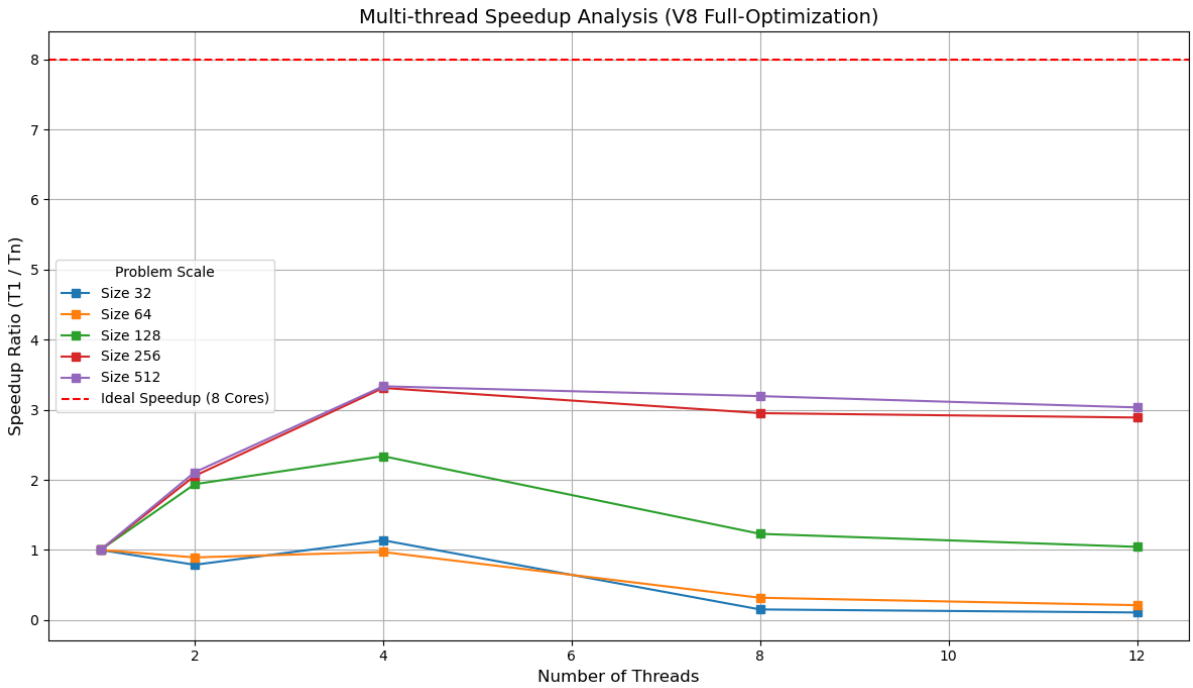

Figure 3: 实验一加速比

从两种环境下的运行结果图片可以看出, 各优化版本的性能较原始版本相比都有了很大的提升

1.7 实验结果分析

- 单线程算法性能分析:
  - 现象: 随着矩阵规模 N 的增大，所有算法的执行时间呈指数级增长。在小规模（32-128）下，各算法差距较小；但在大规模（512-1024）下，Naive (i-j-k) 版本的斜率明显高于其他版本，而 IKJ 和 Loop Unrolling 版本表现出明显的性能优势
  - 原因:Naive 版本在最内层循环访问 B 矩阵时跨度大, 空间局部性差, 导致严重的 Cache Miss。IKJ 通过交换循环次序, 使最内层循环对矩阵 B 的访问变为连续内存空间，极大提高了空间局部性，程序运行效率大大提升。
- 多线并行与加速比分析:
  - 现象:
    * 大计算规模优势: 当规模达到 1024×1024 时, 随着线程数增加, 加速比呈现明显的上升趋势。
    * 小规模" 反向优化": 在 $N = 32$ 或 64 时, 增加线程数不仅没有提速, 加速比反而小于 1(即变慢)。
    * QEMU 瓶颈: 在线程数接近或超过 8 时 (-smp 8), 加速比曲线趋于平缓, 甚至在 $T = 12$ 时略有下滑。

    - WSL 线程与核心数权衡:WSl 中当线程数为 12(大于核心数 6) 时, 运行效率低于线程数为 6 的情况。
  - 原因:
    - 线程开销控制: 小规模任务中, 计算任务本身极快 (微秒级), 而多线程的创建、同步和销毁开销远超计算量, 导致效率下降，此时多线程体现不出优势。
    - 模拟器资源争抢：QEMU 或 WSL2 Arch linux 在宿主机上运行，当模拟线程数超过宿主机物理核心（6 核）或分配核心（8 核）时，会产生剧烈的上下文切换开销，导致系统负载过载。
- 仿真环境与真实环境差异:
  - 现象:WSL 宿主机上的运行时间比 QEMU 低约 2 各数量级;WSL 环境下的曲线更为平稳且符合理论预期;QEMU 下的曲线存在细微波动，且在处理超大规模矩阵时会出现系统调用报错或计时器延迟。
  - 原因:QEMU 作为系统级模拟器，需要将每一条 RISC-V 指令动态翻译为 x86 指令执行，这带来了巨大的软件模拟损耗。同时宿主机拥有真实的硬件缓存和预取器（L1d 288KB），而 QEMU 的缓存层级是软件定义的，无法完美重现硬件级别的访存延迟。
- 其他情况分析:
  - 现象
    - 矩阵转置优化效果并不是很好。
    - 矩阵分块算法优化随着分块的增大, 程序性能可能会降低。
  - 原因
    - 矩阵转置优化版的代码要先执行矩阵优化, 这一步会拖慢矩阵乘法的计算, 即使之后能使得矩阵 B 的访问也是内存连续访问。
    - 当矩阵分块大小超过 L1/L2cache 时, 会使得即使是分块计算矩阵乘法也会出现较大的 Cache Miss 率, 且矩阵分块乘法算法本身较为复杂, 所以实际使用时分块优化的效果也不是最好的。

1.8 实验结论

实验结果证明, 即使是简单调整矩阵乘法内层访问顺序, 使得对所有矩阵都是内存连续式访问, 就能极大增加程序运行效率, 充分体现了空间局部性对程序性能的深刻影响。各种优化方法中循环重排、多线程并行是提升矩阵乘法性能的核心手段, 其中取得最佳优化结果的就是 6 线程并行优化。在多线程应用中，应根据计算规模选择合适的线程数，避免因过载导致的“过饱和”性能损耗, 使得多线程的优势被线程切换等的损耗给掩盖。同时，QEMU 仿真环境能有效验证算法逻辑和相对趋势，但其绝对性能指标受限于软件翻译和宿主机核心资源。